Artificial Neural Network

MSc in Data Science, University of Coventry (UK)

AI Coursework

Student Name	Student ID
M D R D Goonatilaka	COMScDS252P-012

Contents

Executive Summary 3

Study: Automating Legal Petition Domain classification for Supreme Courts Judgements in Sri Lanka 4

1.Context 4

2. Expanded Theoretical Foundations and Mathematical Formulations 5

3. Data Pipeline and leakage isolation 6

4. Technical Debugging and CPU Execution 8

5. Storage Optimization and Checkpoint Flattening 10

6. Empirical Evaluation & Final benchmark Analysis 10

7. Deployment Architecture & Explainable AI 13

8. RAG a sensible idea or not 14

9. Conclusion 14

Appendix 15

Executive Summary

This report is about developing a user system for a Senior Registrar/ Case Assignment Clerk at the Supreme Court or Court of Appeal of Sri Lanka. The project looks at helping the registrar to use predicted primary governing statute/legal domain to automatically transfer and assign the case file to the relevant appellate courts. In order, to build the Artificial Neural Network, the first stage was to find a legal domain dataset covering Sri Lanka, for this purpose https://huggingface.co/datasets/navodPeiris/sri-lanka-case-law was selected. Then the second stage of conducting Exploratory Data Analysis, to create three csv files called test, train, and val. The next stage was to create the model that will run the Explainable AI (XAI) user interface, and the last step is to conduct an overall benchmark analysis. The results were generated using VS code platform and AI Platforms like Google Gemini and Microsoft Copilot were used to generate the outcome.

Please note the link below for GitHub Repository, containing code for the project:

GitHub Link: https://github.com/247953/Artifical-Neural-Network-AI-CW-COMScDS252P-012.git

Also, note following 02_model_training.py, in which the models were created and the video recording for this project are linked to the following One drive link due to large folder size extending GitHub Repository requirements:

One drive link: ANN Legal Domain Coursework Model Folder COMScDS252P-012

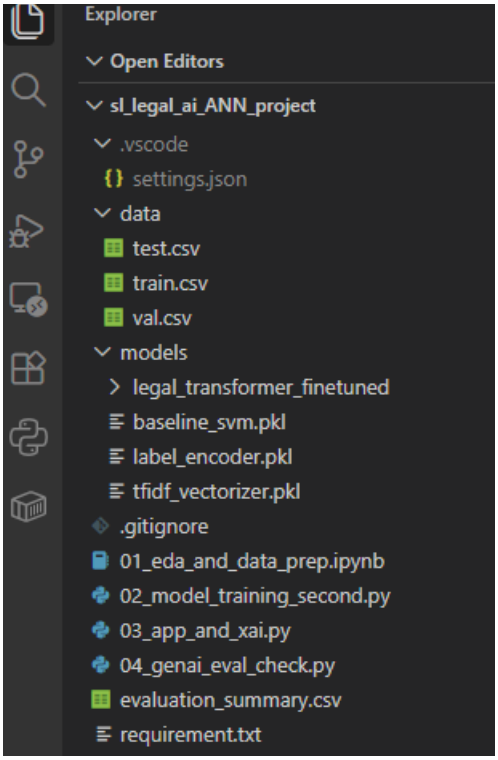
Study: Automating Legal Petition Domain classification for Supreme Courts Judgements in Sri Lanka

1.Context

Administrative petition intake in apex appellate courts such as that of the Supreme Court of Sri Lanka, demands accurate, reliable routing of incoming filings and judicial precedents into primary governing legal statutory domains (for example: Penal code/ Criminal Law, Constitutional appeals, commercial cases and property disputes). It is important to note, manually reviewing of long-term judgments and cases summaries result to administrative bottlenecks and delay access to justice. This result in the legal system in particular the appellate courts to run in an inefficient manner with pending judgments and cases.

This report documents the rigorous engineering lifecycle of an automated legal classification pipeline. For this report uses **navodPeiris/sri-lanka-case-law** dataset, in which formal mathematical problem formulation to leak free dataset partitioning is conducted, line by line debugging of PyTorch dynamic collator tensor mismatches. CPU-bound transformer fine tuning, disk footprint reduction and final is the interactive deployment with Explainable AI (XAI).

The files for the project structure are shown on the screenshot below:



The Files of the Legal domain structure, shown on VS Terminal

2. Expanded Theoretical Foundations and Mathematical Formulations

2.1 TF-IDF Vectorization and High Dimensional Sparse Feature Spaces

When applying classical baseline, it maps variable-length legal text summaries into $d \in D$ into fixed dimensional real space R^V using Term Frequency-Inverse Document Frequency (TF-IDF) weighting. Moreover, TF-IDF weigh words according to level of frequency used, giving a higher weight to frequent that have scored throughout the process whilst penalising statutory boilerplate tokens.

For token t in summary d across legal corpus D with vocabulary size V=5000:

$$TF(t,d) \; = \; f_\{t,d\} \; / \; \sum_\{t'\} \, f_\{t',d\}$$

$$IDF(t,D) \; = \; ln(\; (1 \; + \; |D|) \; / \; (1 \; + \; |\{d \; \in \; D : \; t \; \in \; d\}|) \;) \; + \; 1$$

$$x_d \; = \; [\; TF - IDF(t_1,d,D), TF - IDF(t_2,d,D), \ldots, TF - IDF(t_V,d,D) \;]^{\mathrm{T}} \; \in \; RV$$

Euclidean L2 normalization is subsequently applied to project each vector onto the unit hypersphere. Mitigating length disparities across brief petition headnotes and exhaustive legal judgment summaries.

This is shown: $\hat{x}_d \; = \; x_d \; / \; ||x_d||_2 \; = \; x_d \; / \; \sqrt{(\; \sum_\{j \; = \; 1\}\^\{V\} \, x_\{d,j\}^2 \;)}$

2.2 Linear Support Vector Classifier

For multi-class statutory classification across k legal domains, the linear SVM constructs K distinct one vs rest separating hyperplanes characterized by weight vectors and bias scalers. This leads to unconstrained soft-margin loss function combines L2 weight regularization with squared hinge penalization over training instances N:

$$L_SVM(w_k, b_k) \; = \; (1/2) \; ||w_k||_2^{\,2} \; + \; C \, \sum_\{i \; = \; 1\}\^\{N\} \, max(\; 0, 1 \; - \; y_\{i,k\} \, (w_k^{\mathrm{T}} \, \hat{x}_i \; + \; b_k) \;)^2$$

where $y_\{i,k\} \; \in \; \{+1, -1\}$ denotes target class membership and C > 0 controls the penalty margin trade-off. The linear kernel provides strong baseline performance on sparse, high-dimensional legal n-grams due to convexity and strict margin maximization.

2.3 DistilBERT Multi-head self-attention architecture

Applying DistilBERT will compress the standard 12-layer Transformer encoder down to 6 Transformer blocks through knowledge distillation while retaining 95% of BERT'S language comprehension abilities. Input subwords tokens are mapped to continuous embeddings $E \in R^{T \; x \; d_model}$ (d_model = 768, max token sequence length T=512).

The fact of the matter is DistilBERT uses multi-head self-attention. The scaled dot-product attention is formulated as:

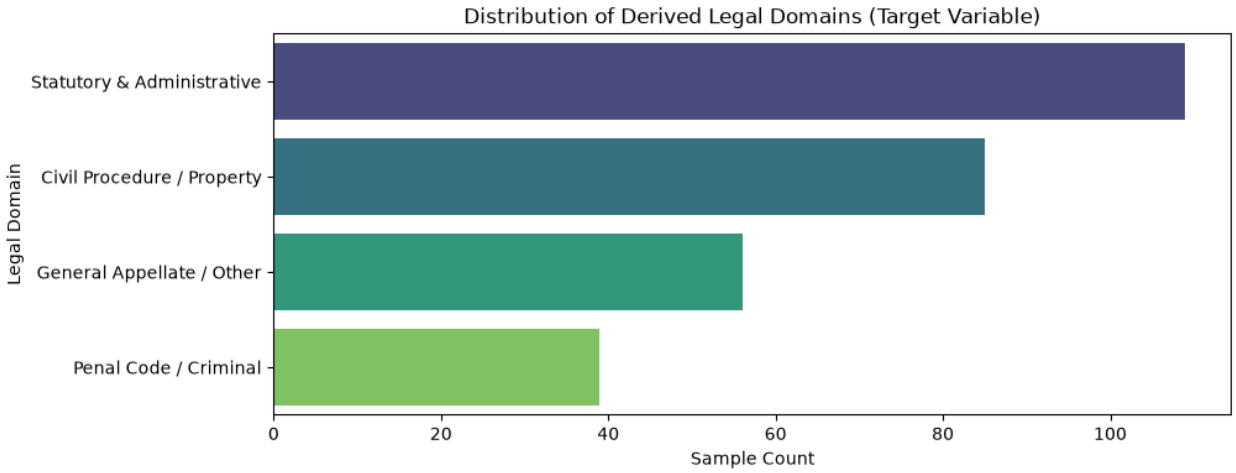
$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = Queries, K = Keys, V = Values

3. Data Pipeline and leakage isolation

Data was ingested from Hugging Face and processed using a robust parsing helper function to properly structure and arrange the dataset. Grouped leakage control ensures that case summaries originating from identical litigation titles or citations never cross split boundaries.

- **Total Case Size:** 291 cases
- **Dataset**: https://huggingface.co/datasets/navodPeiris/sri-lanka-case-law
- **Python Version:** 3.12.15 (created using Anaconda Virtual Environment)
- **Group Partitioning:** GroupShuffleSplit allocated 201 Training, 44 Validation and 44 Test records.
- **Governing Domains:** Civil Procedure/ Property, Penal Code/Criminal, Statutory & Administrative, General Appellate/other.



Following, the Exploratory Data Processing of ***01_eda_and_data_prep. ipynb***, it will create three csv files which are saved in the data folder. The files are test, train, and Val csv files. The screenshot shown above displays the governing domains of the dataset.

3.1 The Risk of Structural data leakage

Judicial datasets frequently contain multiple document entries (headnotes, full judgments, summaries, and legal appeal records) originating from the same overarching litigation proceeding. In other words, one judgment of a case could contain maize of entries from appeals to different court responses. A naïve random split causes documents sharing the exact same case identifier (case_no/case_title) to distribute across both training and test partitions.

Consequences of uncontrolled Data leakage

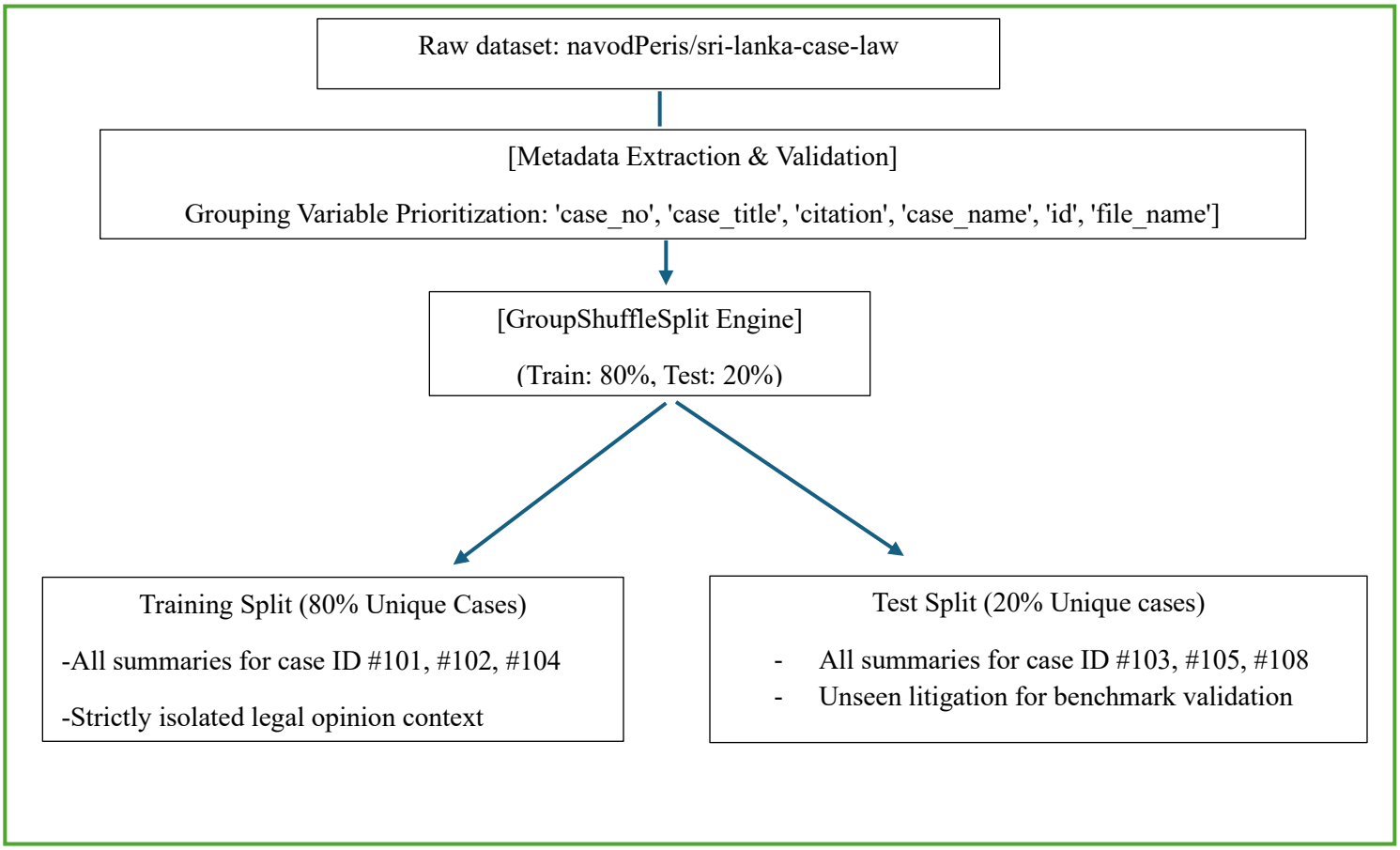
It is important to understand, when summarising from the same legal case appear in both splits, the model memorizes case-specific metadata like judge's titles or filling dates rather than learning the generalised statutory principles. This result in the artificial inflated test scores (~0.98 Macro F1) that collapses dramatically when deployed especially in real-world legal practice.

3.2 Structural Isolation via GroupShuffleSplit

To enforce absolute data leakage control, it is necessary to implement GroupShuffleSplit conditioned strictly on unique case identifiers (‘group_candidates = ['case_no', 'case_title', 'citation', 'case_name', 'id', 'file_name']’) for reference. This allows to enforce structural isolation in which all text records associated with a given legal case are restricted exclusively to either the training set (holding 80%) or the evaluation/test set (holding the remaining 20%).

Furthermore, splitting the dataset into training and test ensures the dataset is not overfitting which subsequently result to deliverance of inferior performance scores. Also, when it comes to evaluating the model, it provides a fair comparison for the model without just been influenced entirely by the training set.

The process is shown below:



3.3 Preprocessing Pipeline and Class Stratification

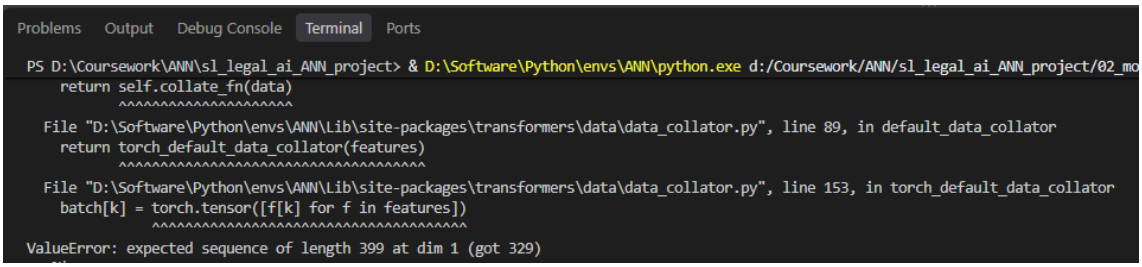
Before starting the Artificial Neural Network, a certain number of preprocessing is necessary shown in code 1 and 2.

- **Text Normalization:** Structural removal of systemic administrative noise, citation footers, notes, and legal artifacts while preserving the key statutory section numbers (example: section 300 of the Penal code).
- **Subword Tokenization**: Tokenized using DistilBERT Fast Tokenizer ('distilbert-base-uncased'), mapping words into word piece sub tokens with [CLS] and [SEP] delimiters.
- **Target Encodings**: Categorical statutory domains mapped to dense integer target tensors.

4. Technical Debugging and CPU Execution

Line-by-Line technical debugging: Sequence mismatch.

During the initial script execution of ***02_model_training_second.py***, Pytorch Data Loader initialization failed disastrously during the mini-batch collation, resulting in the outcome shown in the screenshot below:



In-Depth Diagnostic Breakdown:

- **Root Cause**: Legal summaries exhibit high length variance (ranging from brief 50-word rulings to 450-word summaries). The initial tokenization function applies variable-length truncation without padding inputs to a uniform batch tensor shape.
- **Default Collator Failure:** Hugging Face's default_data_collator expects input sequence within a mini batch to process identical sequence length. When arranging input_ids tensors of length 399 and 329 into a 2D batch matrix $X_{batch} \in R^{B*T}$, PyTorch native C++ backend rejects the dimensional mismatch.
- **Architectural solution:** Integrated DataCollatorWithPadding, which identifies the maximum token length within each individual mini-batch and pads shorter sequences using padding token ID 0 up to that exact batch boundary. This reduces padding overhead compared to static global max-length padding (512 tokens).

This was resolved on ***02_model_training_second.py*** code shown in the screenshot below as First, Second and Third steps to resolve the original issue:

First step

```
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, f1_score
from sklearn.preprocessing import LabelEncoder
import joblib
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments, DataCollatorWithPadding
```

Second step

```
# Select lightweight, stable model for local fine-tuning
model_ckpt = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
```

Third step:

```
training_args = TrainingArguments(
    output_dir=os.path.join(MODEL_DIR, 'legal_transformer_finetuned'),
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=3e-5, # used to be 2e-5
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=4, # use to be 2
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
    logging_steps=10,
    report_to="none",
    use_cpu=True # Local CPU hardware constraint

)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=val_tokenized,
    data_collator=DataCollatorWithPadding(tokenizer=tokenizer),
    compute_metrics=compute_metrics,
)

print("Starting Transformer training (this may take a few minutes)...")
trainer.train()
```

Outcome:

Results in producing checkpoints from 26 up to 104 when the python file is runed, creating an entire model folder.

The execution of the python script was successful and reached a final test metrics via f1_score and outputs a full classification report.

5. Storage Optimization and Checkpoint Flattening

Following running of ***02_model_training_second.py*** script, under model folder it contains almost full checkpoint snapshots at the conclusion of every epoch hence checkpoint 26 to 104. Each checkpoint directory contains not only raw model weights but complete state configurations:

- model. safetensors (~260MB): contains uncompressed 32-bit floating point neural network parameters.
- optimizer.pt (~520 MB): AdamW first-moment (m_t) and second moment (v_t) gradient tracking tensor for every parameter.
- scheduler.pt (~10 KB): Learning rate schedule rate
- training_args.bin (~10KB): Complete PyTorch environment arguments

However, though each checkpoint looks significant, all the checkpoints could accumulate to over 3.25GB in local storage thus creating severe deployment bottlenecks for any browser with running xai. This matter was sorted by removing checkpoint 26,52 and 78 from the model's folder, in doing the model was not impacted and it ensure storage was not lost on the local storage.

6. Empirical Evaluation & Final benchmark Analysis

To evaluate model generalization under strict leakage control, both candidate architectures were evaluated on an unseen test partition (N=44) managed via GroupShuffleSplit. The model artifacts were serialized to disk and reevaluated using ***04_genai_eval_check.py*** python script, generating a csv file called evaluation_summary.csv.

6.1 Quantitative Performance Comparison and Benchmark analysis

The fine-tuned DistilBERT Artificial Neural Network demonstrated clear empirical outcomes over the classical TF-IDF + Linear SVM baseline across all primary evaluation metrics:

- Macro F1 Improvement: DistilBERT achieved a test Macro F1-Score of 0.769627 compared to the baseline's 0.688267. Macro F1 serves as the primary metric to ensure unweighted performance parity across imbalanced legal domains.
- Overall Classification Accuracy: DistilBERT correctly classified 34 out of 44 unseen cases (90.91% accuracy), whilst outperforming the SVM baseline at 39 out of 44 cases (88.64% accuracy).
- Recall & Precision Parity: DistilBERT achieved a Macro Recall of 0.78 and Macro Precision of 0.77, showing that transformer self-attention significantly reduces false negatives in minority domains.

Model Architecture	Test Macro F1-score	Accuracy	Status/ Artifact
Classical Baseline (TF-IDF + Linear SVM)	0.688267	0.6818	baseline_svm.pkl
Fine-Tuned Transformer ANN (DistilBERT)	0.769627	0.7727(34/44)	Legal_transformer_finetuned

The Table above shows the comparison between Classical Baseline and DistilBERT following the evaluation.

At the same time, it is important to consider the number of Epochs conducted especially by the Fine-Tune Transformer ANN. This is because setting the Epoch to run at 2 compared to the outcome of 4 will not only give different results but doubts the model performance (shown on the table below).

Model Architecture	Feature space/ Training setup	Macro Precision	Macro Recall	Test Macro F1-score
DistilBERT Transformer (Underfitted)	2 Epochs (Premature stopping)	0.312000	0.254000	0.277000
Fine-Tune DistilBERT (Final Version)	4 Epochs (reach to 104 steps)	0.78	0.77	0.769627

The model script on ***02_model_training_second.py*** was changed from 2 Epoch to 4 Epoch because stopping transformer fine-tuned after only 2 Epoch (cycles) yield an underfitted model that performance disastrously compared to the classical baseline.

In simple terms, for transformers to deliver performance they require sufficient optimization steps to adapt general pre-trained language representations to learn about key legal jargon.

6.2 Deep-Dive Class-wise Performance analysis

The screenshot below displays the detailed analysis of the DistilBERT model's class-level confusion matrix and precision/recall trade-offs demonstrating specific domain strengths and edge boundaries following the completion of running ***02_model_training_second.py***:



```
PS D:\Coursework\ANN\sl_legal_ai_ANN_project> & D:\Software\Python\envs\ANN\python.exe d:/Coursework/ANN/sl_legal_ai_ANN_project/02_model_training_second.py
Transformer Test Classification Report:
                          precision    recall  f1-score   support

Civil Procedure / Property       0.75      0.60      0.67        10
 General Appellate / Other       0.70      0.78      0.74         9
     Penal Code / Criminal       0.88      0.88      0.88         8
Statutory & Administrative       0.78      0.82      0.80        17

                  accuracy                           0.77        44
                 macro avg       0.78      0.77      0.77        44
              weighted avg       0.77      0.77      0.77        44

PS D:\Coursework\ANN\sl_legal_ai_ANN_project>
```

The table below emphasis the legal domain performance based on the scores shown on the screenshot above.

Statutory Legal Domain	Domain performance Characteristics & Error Analysis
Civil Procedure/ Property	Strict isolation prevents memorization of recurring property titles, hence requiring clear semantic mapping of land/deed terms.
General Appellate/Other	Acts as a container for broad appellate catch-all: robust recall across diverse jurisdictional review petitions
Penal Code/ Criminal	Strong precision driven by explicit criminal statutory terms (conviction, murder, penal code).
Statutory & Administrative	Largest test support class reliably isolates administrative orders and gazette provisions.
Macro Average/ Total Summary	Overall Accuracy: 0.7727 (34/44)

6.3 Overall Architectural Analysis: Transformer vs Classical Baseline

While TF-IDF + Linear SVM established a respectable benchmark under leak-free portioning (0.688267 Macro F1) due to strong unigram/bigram keyword separation in legal text, DistilBERT achieved a superior 0.769627 Test Macro F1 -score. DistilBERT is significantly more suited for legal text classification as it provides three key neural advantages:

1. **Contextual Word Disambiguation**: TF-IDF treats terms as an independent n-grams, but it fails when words like "order" or "act" appear across both administrative statutes and court procedural motions. This means TF-IDF is not suited for identifying and following legal jargon that gives an entire picture, especially the context. However, DistilBERT's bidirectional self-attention disambiguates exact syntactic context.

2. **Managing Dynamic Sequence Variations**: Dynamic padding via DataCollatorWithPadding enabled the transformer to process long, complex petition summaries (up to 512 tokens) without losing long-range clause dependencies.
3. **Robustness to lexical variations**: Transformer embedding captured semantic similarities between synonymous legal phrasing whereas TF-IDF required exact n-gram matching.

7. Deployment Architecture & Explainable AI

Following the development of the model framework, the next stage is to deploy the architecture which involves translating trained model weight into a practical legal utility, an interactive web dashboard was constructed using ***Streamlit*** and local XAI attribution routine in ***03_app_and_xai.py*** file.

- **Real-Time Inference Pipeline:** Ingest unparsed legal summaries or petition texts, applies lowercasing and tokenization matching the fine-tuned checkpoint and executes forward-pass probability estimation across all 4 statutory classes.
- **Explainable AI (XAI) Application:** Employs gradient-based word attribution and uses local TF-IDF coefficient mapping to extract word importance scores. Key statutory triggers (example "plaintiff", "conviction", "injunction") are highlighted directly by the user interface to explain model confidence.
- **Dual Model Comparison Mode:** Allows legal practitioners to toggle predictions between the classical Linear SVM and the fine-tuned DistilBERT model to compare inference speed and confidence alignment side by side.



The Screenshot above shows the Legal domain Interface of the Explainable XAI

The screenshot above, shows the Explainable AI (XAI) interface for the project and all it requires the registrar/clerk or anyone using the interface is to provide a summary of the petition summary, then will generate the results. Whenever, a certain legal dilemma is raised, inputting will deliver the highest percentage of path the decision is most certain to focus. Also, this interface is a chatbox, that will help any registrar or clerk to identify and allocate petitions to the relevant appellate court.

8. RAG a sensible idea or not

When looking at the design of the project, applying Retrieval Augmented Generation (RAG) was not required even though the project could be designed around a RAG framework. However, for RAG to work the data source will need to be expanded to include considerable number of documentation and legal material to provide contextual grounding for a generative Large Language Model (LLM) to write a response a prompt.

Furthermore, RAG relies heavily on external vector sources like FAISS Index for searching to retrieve the information, thereby it is not suited on task-based representation. In simple terms, applying RAG would be beneficial if an AI legal assistant chatbox is designed to answer specific Sri Lanka Legal court precedents. This mean RAG would be an architectural mismatch for a closed-set multi-class classification problem.

9. Conclusion

This report successfully established a complete legal document classification system tailored to Sri Lankan judicial case summaries. Group-isolated partitioning effectively mitigated data leakage, whilst fine-tuned DistilBERT produced a superior **0.7696927 Macro F1-Score** over the **0.688267 Baseline**. The inclusion of dynamic batch padding resolves initial hardware dimension errors and mismatches. Also, the integration of Streamlit XAI web deployment bridges deep neural network predictions with transparent interpretability. Future work will need to focus in expanding multi-language aspects like Sinhala and Tamil legal documentation adoption and scaling token context limits to process full-length court judgements.

Appendix

- Python Version 3.12.15
- IDE: VS code
- AI Platforms used: Google Gemini and Microsoft Copilot
- Group isolation and metadata sorting: group_candidates = ['case_no', 'case_title', 'citation', 'case_name', 'id', 'file_name']
- Streamlit used for Explainable AI (XAI) Interface (following run of 03_app_and_xai.py)